IntelliDoc

AI-Powered Document Q&A Platform

Gaurav Jangir

October 10, 2026

Abstract

IntelliDoc is a high-performance, production-ready Retrieval-Augmented Generation (RAG) application. It bridges the gap between unstructured document data (such as PDFs) and actionable insights by allowing users to query documents using natural language. Leveraging dense vector search and Google's Gemini Large Language Model, IntelliDoc provides accurate, factually grounded answers with transparent source citations.

Contents

1 1. The Idea **2**

2 2. Planning & Requirements **2**

3 3. Architecture **2**

4 4. Implementation Details **3**
4.1 Document Ingestion Pipeline 3
4.2 Embedding and Indexing 3
4.3 Retrieval-Augmented Generation (RAG) 3

5 5. Advantages & Key Features **3**

6 6. Results and Outputs **3**

7 7. Conclusion & Future Scope **4**

1 1. The Idea

With the exponential growth of digital documents, extracting specific, accurate information from lengthy PDFs has become a time-consuming challenge. Traditional keyword search is often insufficient because it lacks semantic understanding. The idea behind IntelliDoc was to build an "Innovation Catalyst" tool—a system that reads, understands, and retrieves information from documents intelligently, acting as a personal research assistant.

2 2. Planning & Requirements

The project was designed with a focus on modularity, speed, and accuracy. The core requirements were:

- **High-Fidelity Ingestion:** Ability to parse PDFs, including complex structures like tables, without losing semantic context.
- **Low-Latency Retrieval:** Fast vector similarity search to find relevant information instantly.
- **Factuality & Grounding:** The LLM must not hallucinate; every answer must be strictly derived from the uploaded documents.
- **User Experience:** A clean, intuitive web interface for uploading documents and conducting chat-based Q&A.

3 3. Architecture

IntelliDoc follows a modern, decoupled client-server architecture:

- **Frontend (Streamlit):** Provides an interactive UI for document management and real-time chat. It communicates with the backend via REST APIs.
- **Backend (FastAPI):** Handles all business logic, asynchronous task processing, and API routing.
- **Document Parsing (PyMuPDF4LLM):** Converts PDFs into structured Markdown, preserving tables, lists, and hierarchical structures.
- **Vector Database (FAISS):** An in-memory, high-performance similarity search index optimized for dense vectors.
- **Embedding Model (Sentence-Transformers):** Uses the local `all-MiniLM-L6-v2` model to generate 384-dimensional dense vectors from text chunks.
- **LLM Engine (Google Gemini 3.8 Flash):** Generates human-like answers strictly grounded in the retrieved context passages.

4 4. Implementation Details

4.1 Document Ingestion Pipeline

When a user uploads a PDF, the system calculates an MD5 hash to prevent duplicate processing. The document is parsed into Markdown using PyMuPDF to preserve grid data and tables. The Markdown is then recursively chunked into overlapping segments (chunk size of 1000 characters, overlap of 200 characters) to maintain contextual continuity.

4.2 Embedding and Indexing

The text chunks are passed through a local Sentence-Transformer model. Processing embeddings locally ensures privacy and reduces API costs. The resulting embeddings are indexed using Facebook AI Similarity Search (FAISS) using the `IndexFlatL2` metric, enabling sub-millisecond retrieval times.

4.3 Retrieval-Augmented Generation (RAG)

Upon receiving a user query, the system embeds the query using the same local model. FAISS retrieves the top-K most semantically similar chunks. These chunks are injected into a strict prompt template alongside the user's question, instructing the Gemini LLM to synthesize an answer based *only* on the provided context.

5 5. Advantages & Key Features

- **Prevents Hallucinations:** The system forces the LLM to cite its sources, drastically reducing AI hallucinations.
- **Preserves Tabular Data:** By converting PDFs to Markdown, the system understands structured data (like chemical composition tables) that standard parsers miss.
- **Data Privacy:** Embeddings are generated and stored locally, ensuring sensitive document text isn't sent to third-party embedding APIs.
- **Scalability:** The FastAPI backend is asynchronous and production-ready, capable of handling multiple concurrent requests.

6 6. Results and Outputs

The final system successfully answers complex queries spanning multiple pages of a document. In testing with technical metallurgy and materials science documents, the system accurately retrieved percentage compositions from embedded tables (e.g., identifying that PbO_2 has an 86.7% lead composition).

Every response generated by IntelliDoc includes a drop-down section detailing the exact retrieved sources, displaying the origin file name, page number, and similarity score, thereby building trust with the end-user.

7 7. Conclusion & Future Scope

IntelliDoc demonstrates the power of combining traditional software engineering with modern generative AI. It solves a real-world problem effectively and efficiently. Future improvements could include integrating multimodal capabilities to analyze purely visual data (like pie charts and graphs) and expanding support for additional file formats such as Word documents and Excel spreadsheets.